



교통 표지판 이미지 분류 (TensorFlow) - code 포함

차량 운전자의 기본 소양은 내 차의 주변 상황을 실시간으로 인지하면서 시시각각 변하는 도로 상황 속에서 사고가 나지 않도록 안전하게 운전하는 것일 것입니다. 이 주변 상황을 객관적으로 알려주는 장치에는 **교통 표지판**이 있을 것입니다. 교통 표지판은 앞선 도로의 상황이 어떨 것인지, 해당 도로에서는 어떤 점을 주의해야 하는지 등의 정보를 담고 있기에 초보 운전 때부터 모든 내용을 숙지하고 도로로 나오는 것이 바람직할 것입니다.



최근에는 인공지능 기술이 발전함에 따라 사람이 아닌 자동차가 스스로 운전할 수 있게 하는 자율주행 기술이 함께 발전하고 있습니다. 자율주행 기술의 핵심은 역시 자동차에 장착된 카메라가 영상으로 도로 상황을 포착하는 것일 것입니다. 이 영상에는 당연히 교통 표지판도 포함되어 있을 것이므로, **교통 표지판의 내용을 분류**한 다음에 자동차가 적절한 행동을 취하는 과정이 필요할 것입니다.

따라서 이번 프로젝트에서는 교통 표지판이 촬영된 이미지를 통해 교통 표지판이 어떤 내용인지 분류하는 모델을 딥러닝 모델을 통해 간단하게 구현할 것입니다.

이를 위해 이미지 데이터 처리에 특화된 **Convolutional Neural Network(CNN)** 를 활용할 것입니다.

프로젝트 목표

- 이미지 데이터를 전처리하는 과정과 딥러닝 모델에 적용하기 위한 형태로 변환하는 과정을 학습
- 교통 표지판 이미지 데이터를 활용하여 CNN 모델로 표지판 종류를 예측하는 분류 모델 학습

데이터 출처

- <https://www.kaggle.com/meowmeowmeowmeowmeow/gtsrb-german-traffic-sign>

프로젝트 목차

1. 데이터 확인하기: 프로젝트에 사용할 데이터셋 디렉터리가 어떤 구조를 가지는지 확인
2. 데이터 분석: 데이터셋에 포함된 이미지 데이터셋들이 어떤 특징을 가지는지 분석
3. 데이터셋 구성: 학습과 평가에 사용할 데이터셋을 클래스로 정의
4. 모델 학습: 데이터셋을 이용해 간단한 CNN 모델을 학습 후 성능 평가
5. 모델 성능 개선(TODO): 모델의 성능을 향상하기 위해 다양한 방법 적용

1. 데이터 확인하기

```
import os
import random

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
import tensorflow as tf
from PIL import Image, ImageDraw

# Tensorflow 관련 디버그 및 경고 메시지 비활성화
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"

SEED = 2022
tf.random.set_seed(SEED)
tf.keras.utils.set_random_seed(SEED)
tf.config.experimental.enable_op_determinism()
```

```
np.random.seed(SEED)
random.seed(SEED)
```

데이터 구조 확인하기

본 프로젝트에서 사용할 데이터셋은 data라는 디렉터리에 아래와 구조로 저장되어 있습니다.

```
data
├── Train
├── Train.csv
├── Test
├── Test.csv
├── Meta
├── Meta.csv
```

Meta 및 **Meta.csv**의 내용은 본 프로젝트에서는 사용할 필요가 없기 때문에 따로 다루지 않습니다.

Train.csv 및 **Test.csv**에서는 각각 **Train** 폴더의 이미지와 **Test** 폴더의 이미지들의 메타 데이터를 담고 있습니다.

바로 확인해 보겠습니다.

```
root_dir = "/mnt/elice/dataset"

train_metadata = pd.read_csv(os.path.join(root_dir, "Train.csv"))
test_metadata = pd.read_csv(os.path.join(root_dir, "Test.csv"))

train_metadata.head()
```

```
test_metadata.head()
```

두 파일 모두 같은 컬럼을 가지고 있는 것을 확인할 수 있습니다. 각 컬럼의 내용은 다음과 같습니다.

- **Width**: 이미지 파일의 너비
- **Height**: 이미지 파일의 높이

- `Roi.X1`: 이미지 파일 내에서 실제 표지판이 존재하는 곳을 상자로 감쌌을 때 좌상단 X 좌표
- `Roi.Y1`: 이미지 파일 내에서 실제 표지판이 존재하는 곳을 상자로 감쌌을 때 좌상단 Y 좌표
- `Roi.X2`: 이미지 파일 내에서 실제 표지판이 존재하는 곳을 상자로 감쌌을 때 우하단 X 좌표
- `Roi.Y2`: 이미지 파일 내에서 실제 표지판이 존재하는 곳을 상자로 감쌌을 때 우하단 Y 좌표
- `ClassId`: 해당 이미지의 클래스 ID (0 ~ 42까지 존재)
- `Path`: 실제 이미지가 저장된 경로

`Roi` 컬럼의 내용은 이미지로 직접 확인해 보면 아래와 같습니다. (Roi는 Region of Interest라는 뜻입니다.)

```
sample_metadata = train_metadata.iloc[28589, :]
sample_img = Image.open(os.path.join(root_dir, sample_metadata["Path"]))

roi_box = ImageDraw.Draw(sample_img)
roi_box.rectangle(
    (
        sample_metadata["Roi.X1"], # 좌상단 X 좌표
        sample_metadata["Roi.Y1"], # 좌상단 Y 좌표
        sample_metadata["Roi.X2"], # 우하단 X 좌표
        sample_metadata["Roi.Y2"], # 우하단 Y 좌표
    ),
    outline=(255, 0, 0), # Bounding Box 색을 빨간색으로
    width=2, # Bounding Box 선의 두께
)

plt.imshow(sample_img)
```

설명한 대로 표지판을 감싸는 Bounding Box를 그리는 것을 확인할 수 있습니다.

이 정보는 본 프로젝트의 목적인 이미지 분류에는 크게 필요하지 않지만, 객체 인식(Object Detection) 작업을 할 때는 필수적인 정보입니다.

02. 데이터 분석

데이터 셋 특징 확인

이제 CSV로 저장된 메타데이터를 통해 전체 데이터 셋에 있는 이미지 파일들이 어떤 특징을 가지는지 확인해 보겠습니다.

먼저 앞서 메타데이터를 확인해 본 결과 각 이미지 파일의 너비와 높이가 모두 다른 것처럼 보였는데, 좀 더 확인해 보겠습니다.

```
train_metadata.head(n=20)
```

역시 모두 같은 크기가 아님을 알 수 있습니다. 딥러닝 모델에 적용하기 위해서는 모두 같은 크기의 이미지를 가져야 하므로 하나의 값으로 통일하는 것이 좋습니다.

그러기 위해 먼저 학습 데이터에서 이미지 너비(Width)와 높이(Height)의 분포를 살펴보겠습니다.

```
fig, ax = plt.subplots(2, 1, figsize=(12, 8))
sns.set_palette("tab10")
sns.histplot(
    train_metadata,
    x="Width",
    kde=True,
    stat="percent",
    label="Width",
    ax=ax[0],
    color="r",
)
ax[0].legend()
sns.histplot(
    train_metadata,
    x="Height",
    kde=True,
    stat="percent",
    label="Height",
    ax=ax[1],
    color="b",
)
ax[1].legend()
```

```

print(
    f"[너비] 평균: {train_metadata['Width'].mean()}\n"
    f"[너비] 중간값(median): {train_metadata['Width'].median()}\n"
    f"[너비] 표준편차: {train_metadata['Width'].std(ddof=0)}"
)
print("=" * 50)
print(
    f"[높이] 평균: {train_metadata['Height'].mean()}\n"
    f"[높이] 중간값(median): {train_metadata['Height'].median()}\n"
    f"[높이] 표준편차: {train_metadata['Height'].std(ddof=0)}"
)

```

이미지 사이즈 정의

대부분의 이미지 파일의 너비와 높이는 50 이하인 것으로 확인됩니다. 실제로도 중간값은 50보다 작은 값에 위치하기도 합니다.

이미지 크기를 하나로 통일하기 위해서 어느 정도 절충안이 필요해 보이는데, 이때는 보통 유명한 모델이나 데이터 셋이 사용하는 값을 사용하는 것이 일반적입니다.

대표적인 이미지 사이즈로는 224x224, 32x32 등이 있습니다. 이번 프로젝트에서는 32x32로 이미지 크기를 통일하겠습니다.

```
img_height, img_width = 32, 32
```

03. 데이터셋 구성

이제 Tensorflow를 활용하여 모델 학습을 위한 학습 데이터셋과 테스트 데이터셋을 구성하도록 하겠습니다.

먼저 학습 데이터셋은 `Train` 디렉터리에 존재하며, 각 이미지는 클래스 ID 별 디렉터리에 나눠져서 저장되어 있습니다.

테스트 데이터 셋은 하나의 디렉터리에 모든 이미지 파일이 저장되어 있습니다.

두 데이터 셋의 구조가 약간 다르지만, csv 파일을 통해 이미지 파일의 경로와 클래스 ID를 얻을 수 있기 때문에 이를 활용하여 데이터 셋을 구성할 수 있습니다.

데이터셋 클래스 정의

클래스 ID 별로 디렉터리가 나뉘진 경우, Tensorflow에서는 `ImageDataGenerator` 를 활용하면 데이터셋을 손쉽게 생성할 수 있습니다.

구체적인 사용 방법은 먼저 `ImageDataGenerator` 객체를 생성한 후, 이 객체의 `flow_from_directory` 메소드를 통해 데이터셋을 생성하는 것입니다.

클래스와 메소드의 의미를 부연하자면 `ImageDataGenerator` 란 이름에서 알 수 있듯이 이미지 데이터를 **생성(generate)** 하기 위한 클래스입니다. 구체적으로 Python에는 Generator라는 것이 존재하는데, 이것을 의미합니다.

그리고 나서 `flow_from_directory` 는 디렉토리 구조에서 파일들을, generator를 통해 모델로 **흘려(flow)** 보내겠다는 의미입니다.

그러면 코드를 통해 확인해 보겠습니다.

```
# rescale은 입력된 이미지 데이터의 각 픽셀값들을 지정한 수치로 나눠
# 주겠다는 의미입니다.
# 즉 정규화를 의미하는데, 이미지 픽셀은 0 ~ 255의 값을 가지므로 255
# 로 나눠줍니다.
train_gen = tf.keras.preprocessing.image.ImageDataGenerator
(
    rescale=1.0 / 255, validation_split=0.2
)

# flow_from_directory를 통해 데이터셋을 생성합니다.
# 클래스 ID별로 저장된 디렉토리 경로명을 처음에 지정해주어야 합니다.
train_set = train_gen.flow_from_directory(
    os.path.join(root_dir, "Train"),
    target_size=(img_height, img_width),
    class_mode="categorical",
    batch_size=128,
    shuffle=True,
    seed=SEED,
    subset="training",
)

valid_set = train_gen.flow_from_directory(
```

```

os.path.join(root_dir, "Train"),
target_size=(img_height, img_width),
class_mode="categorical",
batch_size=128,
shuffle=False,
subset="validation",
)

```

다음으로 테스트 데이터셋을 만들겠습니다.

테스트 이미지들은 `Test` 디렉터리에 있는데, `Train`과는 달리 클래스 ID 별로 구분된 것이 아니라 그냥 이미지들만 나열되어 있습니다.

그래서 `flow_from_directory`는 사용할 수 없습니다.

대신 메타데이터에 해당하는 CSV 파일에 클래스 ID가 적혀있기 때문에 이를 활용하여 `flow_from_dataframe` 메소드를 통해 데이터셋을 생성할 수 있습니다.

```

# 테스트 파일들의 메타데이터를 DataFrame으로 불러옵니다.
test_metadata = pd.read_csv(os.path.join(root_dir, "Test.csv"))

# flow_from_dataframe 메소드가 클래스 ID에 해당하는 컬럼의 데이터
타입이 문자열(str)이길 요구합니다.
test_metadata["ClassId"] = test_metadata["ClassId"].astype(
    str)

# 학습 데이터처럼 ImageDataGenerator를 생성합니다.
test_gen = tf.keras.preprocessing.image.ImageDataGenerator
(rescale=1.0 / 255)

# flow_from_directory에서는 DataFrame의 어떤 컬럼이 입력(X)이고
어떤 컬럼이 라벨(y)인지 지정해야 합니다.
# 본 데이터셋의 DataFrame은 각각 'Path'와 'ClassId'가 이에 해당
합니다.
# 다음으로 각 이미지 파일의 경로는 'directory' argument의 값과 x_
col에 지정한 컬럼의 각 값을 join해서 얻어지는데,
# 'Path' 컬럼이 'Test/00000.png'와 같이 이루어져 있으므로 'direc
tory'에는 데이터셋의 최상위 경로만 지정하면 됩니다.

```

```
test_set = test_gen.flow_from_dataframe(
    test_metadata,
    directory=root_dir,
    x_col="Path",
    y_col="ClassId",
    target_size=(img_height, img_width),
    class_mode="categorical",
    batch_size=128,
    shuffle=False,
)
```

04. 모델 학습

이제 이 데이터를 이용해 TensorFlow에서 간단한 CNN 모델을 정의하고 학습을 진행해 보겠습니다.

간단한 CNN 모델 정의

Tensorflow에서 Convolutional Layer를 사용하는 방법은 Dense layer를 사용하는 방법과 크게 다르지 않습니다.

다만 `tensorflow.keras.layers.Conv2D`를 사용하면 될 뿐입니다. `2D`가 붙은 이유는 2차원 데이터에 적용하기 위해서입니다.

```
model = tf.keras.Sequential(
    [
        # 특징 추출
        tf.keras.layers.Conv2D(
            filters=5,
            kernel_size=(3, 3),
            activation="relu",
            input_shape=(img_height, img_width, 3),
        ),
        tf.keras.layers.MaxPooling2D((2, 2)),
        # 분류
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(43, activation="softmax"),
    ]
)
```

다음으로 손실 함수 최적화를 위해 Optimizer는 Adam을 사용하며, 손실 함수는 분류 작업이므로 Cross Entropy 함수를 사용합니다.

여기서는 라벨 데이터가 `flow_from_directory`에 의해 알아서 one-hot 인코딩된 형태로 이루어지기 때문에 손실 함수는 `categorical_crossentropy`로 지정합니다.

```
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)
model.compile(
    optimizer=optimizer, loss="categorical_crossentropy", metrics=["accuracy"]
)
```

모델 학습

모델 생성과 컴파일을 모두 마쳤으면, 학습을 진행해 보겠습니다.

```
num_epochs = 5
history = model.fit(train_set, epochs=num_epochs, validation_data=valid_set)
```

```
accuracy = history.history["accuracy"]
val_accuracy = history.history["val_accuracy"]

loss = history.history["loss"]
val_loss = history.history["val_loss"]

epochs_range = range(1, num_epochs + 1)

plt.figure(figsize=(16, 8))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, accuracy, label="Training Accuracy")
plt.plot(epochs_range, val_accuracy, label="Validation Accuracy")
plt.legend(loc="lower right")
plt.title("Training and Validation Accuracy")

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label="Training Loss")
plt.plot(epochs_range, val_loss, label="Validation Loss")
```

```
plt.legend(loc="upper right")
plt.title("Training and Validation Loss")
plt.show()
```

학습 데이터에서는 정확도가 90% 이상으로 높게 나오나, 검증 데이터에서는 성능이 80% 정도로 나오는 것을 확인할 수 있습니다. 테스트 데이터에서는 성능이 어떻게 나오는지 확인해 보겠습니다.

```
_, test_accuracy = model.evaluate(test_set)
print(f"테스트 정확도: {test_accuracy * 100:.3f}%")
```

테스트 데이터에서도 정확도 80%에 조금 못 미치는 성능이 나올 것입니다. 어느 정도 분류를 수행하긴 하나 아직 만족스러운 성능은 아닌 것으로 보입니다.

이는 다양한 원인이 있을 수 있습니다.

- 데이터셋의 크기가 작아서 학습이 제대로 이루어지지 않았을 수 있습니다.
- 학습률이나 다른 하이퍼파라미터가 잘못 설정되었을 수 있습니다.
- 데이터셋이 불균형하게 구성되어 있을 수 있습니다.
- 모델의 구조가 너무 복잡하거나 단순해서 학습이 제대로 이루어지지 않았을 수 있습니다.

그 외에도 다양한 원인이 있을 수 있습니다.

최적의 모델을 얻기 위해서는 다양한 시도가 필요합니다.

모델 성능을 개선하기 위해 앞서, 실제 이미지를 보면서 실제 라벨과 예측값이 잘 일치하는지 몇 개를 샘플로 확인해 보겠습니다.

```
num_sample = 25

test_pred = np.argmax(model.predict(test_set), axis=-1)[:num_sample]
X_test_sample = test_set[0][0][:num_sample, :, :]
y_test_sample = np.argmax(test_set[0][1][:num_sample, :], axis=-1)

plt.figure(figsize=(13, 13))
for i in range(num_sample):
    plt.subplot(5, 5, i + 1)
```

```

plt.grid(False)
plt.xticks([])
plt.yticks([])

prediction = test_pred[i]
actual = y_test_sample[i]
color = "g"
if prediction != actual:
    color = "r"

plt.xlabel(f"Actual={actual} || Pred={prediction}", color=color)
plt.imshow(tf.keras.utils.array_to_img(X_test_sample[i]))

plt.show()

```

5. 모델 성능 개선

모델의 구조를 개선하고 하이퍼파라미터를 조정하여 성능을 높여보겠습니다.

오래 걸리는 학습을 위해 GPU를 사용하여 학습을 진행합니다.

GPU 확인

복잡한 모델을 학습할 때는 GPU를 사용하는 것이 좋습니다. 우선 현재 사용할 수 있는 GPU가 있는지 확인합니다.

```

device_list = tf.config.list_physical_devices("GPU")
print(f"GPU 사용 가능 여부: {len(device_list) > 0}")

```

사용할 수 있는 GPU가 있는 것을 확인했습니다.

[TODO] 제출 파일 생성

여러분의 목표는 모델의 성능을 개선하여 테스트 데이터셋에서 90% 이상의 정확도를 달성하는 것입니다.

3, 4챕터에서 데이터셋과 모델을 구성한 코드를 기반으로, 데이터셋과 모델의 구성을 다각도로 개선하면서 테스트 데이터에 대한 정확도를 90% 이상으로 높여보세요.

4챕터에서 테스트 데이터에 대한 성능을 평가한 코드를 그대로 활용하여 성능을 평가하실 수 있습니다.

```
# [TODO] 데이터셋, 모델을 다각도로 개선하여 정확도를 90% 이상으로  
# 높여보세요.  
  
# 새로운 모델을 구성하고 학습하기 위한 코드를 작성하세요.  
# 기존 모델을 그대로 사용하고, 학습 조건만 변경하셔도 무방합니다.  
new_model = None
```

충분한 성능이 확보되었으면 아래의 형태로 예측값을 만들어 CSV 파일로 제출합니다. 제출 파일 이름은 `submission.csv` 여야 합니다.

Path	label
Test/00000.png	10
Test/00001.png	1
Test/00002.png	32
...	...
Test/12628.png	41
Test/12629.png	27

통과 기준은 **정확도 90% 이상**입니다.

```
# 테스트 데이터셋에서 각 Class ID 별로 할당한 label을 다시 원래 Cl  
# ass ID로 변환하기 위한 inverse map을 구성합니다.  
label_map = test_set.class_indices  
inverse_label_map = {v: k for k, v in label_map.items()}
```

```
# 이 코드를 활용하여 제출 파일을 제작하실 수 있습니다.  
# 제출 파일은 'submission.csv'로 저장됩니다.
```

```
test_set.reset()
```

```
# test_set에 대한 예측 결과를 저장합니다.  
test_pred = np.argmax(new_model.predict(test_set), axis=-1)  
test_pred = [inverse_label_map[label] for label in test_pre  
d]
```

```
test_df = pd.DataFrame(  
    {  
        "Path": test_metadata["Path"],  
        "label": test_pred,  
    }  
)  
  
test_df.to_csv("submission.csv", index=False)
```